PRASAD V POTLURI SIDDHARTHA INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ACADEMIC YEAR: 2026-27

GENERATIVE AI COURSE PROJECT REPORT

TRAVEL PRO AGENTIC AI

An Intelligent India Travel Planning and Seasonal Destination Discovery System

Faculty Name: Dr. Jyothsna K.

Team Members:

1. **Mandadhi Janani** **23501A05B2**
2. **Mukku Dakshayani** **23501A05C5**
3. **L.Lakshmi Poojitha** **23501A05A2**

GitHub Repository Link : https://github.com/MDakshayani/TravelPro_AgenticAI

Project Deployment Link: https://travel-pro-agentic-ai-umber.vercel.app/

Deployment Platform: Vercel

CHAPTER 1: INTRODUCTION

1.1 Background

India is a country with diverse geographical regions, historical monuments, cultural traditions, natural attractions, festivals, and seasonal travel experiences. Selecting a suitable destination requires travelers to consider several factors, including travel duration, destination preferences, weather conditions, location, and the best time to visit.

Traditional travel planning often requires users to visit multiple websites to explore destinations, compare seasonal conditions, view geographical locations, and prepare an itinerary. This process can be time-consuming, particularly for travelers who are unfamiliar with a destination.

Travel Pro Agentic AI is a web-based travel discovery and planning application designed to simplify the process of exploring India and organizing trips. The application provides destination discovery through dedicated sections for Indian states and union territories, a smart months calendar for seasonal exploration, and an Adventure Builder for trip planning.

The application also integrates Google Maps to support geographical exploration and navigation. Live weather information helps travelers consider current or forecast weather conditions when evaluating travel destinations.

The project aims to combine destination discovery, travel planning, geographical information, and weather information in one interface. Its agentic AI aspects should be demonstrated through the actual planning logic, task execution, tool usage, and validation implemented in the application.

1.2 Problem Statement

Travelers planning trips across India often need to gather information from different sources to identify suitable destinations, understand seasonal travel conditions, view locations on maps, and organize their itineraries.

When these activities are performed separately, the planning process becomes fragmented and requires additional time and effort. Travelers may also select destinations without considering current weather conditions or the suitability of a destination during a particular month.

Therefore, the problem addressed by this project is the development of an integrated travel discovery and planning platform that helps users explore Indian destinations, identify seasonal travel opportunities, create trip plans, view geographical information, and access weather information through a unified web application.

1.3 Motivation

The motivation for developing Travel Pro Agentic AI is to make travel planning more convenient, organized, and accessible.

The main motivations are:

- To simplify the discovery of travel destinations across India.
- To provide dedicated exploration of states and union territories.
- To support seasonal destination discovery using a monthly calendar.
- To help users create travel plans based on their preferences.
- To integrate geographical information through Google Maps.
- To provide weather information that supports travel decisions.
- To reduce the need to switch between multiple travel information websites.
- To explore the use of AI-driven planning and external tools in travel-related applications.

1.4 Objectives

The primary objectives of the project are:

1. To develop a web-based platform for discovering Indian travel destinations.
2. To provide an Adventure Builder for creating personalized trip plans.
3. To organize destination information through States and Union Territories sections.
4. To implement a Smart Months Calendar for seasonal travel discovery.
5. To integrate Google Maps for location visualization and navigation support.
6. To provide live weather information to help users evaluate travel conditions.
7. To provide login and registration interfaces for user access.
8. To develop a convenient and organized interface for travel discovery and planning.

9. To apply agentic AI principles wherever supported by the implemented planning workflow.
10. To handle incomplete inputs, unavailable external services, and invalid requests appropriately.

1.5 Scope of the Project

The scope of Travel Pro Agentic AI includes the following features:

Destination Discovery: Users can explore Indian states and union territories through dedicated sections.

Trip Planning: Users can access the Adventure Builder to plan a trip.

Seasonal Travel Discovery: The Smart Months Calendar helps users explore travel opportunities according to the month.

Map Integration: Google Maps provides geographical context and supports the exploration of travel locations.

Weather Information: Weather data helps users consider environmental conditions when planning trips.

User Access: Login and registration pages provide account-related interfaces.

Future Extensibility: The system can be extended with saved itineraries, more travel APIs, advanced preference memory, and additional AI planning capabilities.

The project focuses on travel discovery and planning. Unless separately implemented, it does not claim to provide confirmed hotel bookings, ticket reservations, guaranteed weather predictions, or autonomous purchasing.

CHAPTER 2: LITERATURE SURVEY

2.1 Introduction

The development of an intelligent travel planning application draws on research in language-model reasoning, external tool usage, retrieval-augmented generation, and interactive geographical services.

The literature reviewed below provides a technical foundation for designing systems that can combine language understanding with external information and task execution.

2.2 Review of Existing Research

1. ReAct: Synergizing Reasoning and Acting in Language Models

Authors: Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao.

Year: 2023.

Method: ReAct combines reasoning and actions, allowing language models to interact with external tools or information sources while working toward a task.

Application to Travel Pro: The approach is relevant to breaking a travel request into planning steps, obtaining information through tools, and using the results to produce a travel plan.

Limitation: The method does not automatically guarantee correct external data or successful execution of every task. Tool outputs and final responses still require validation.

2. Toolformer: Language Models Can Teach Themselves to Use Tools

Authors: Timo Schick and co-authors.

Year: 2023.

Method: Toolformer investigates how language models can learn when and how to call external tools through APIs.

Application to Travel Pro: The research supports the use of external services, such as map and weather integrations, alongside language-model capabilities.

Limitation: Tool integration requires appropriate API access, correct arguments, error handling, and reliable interpretation of returned data.

3. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Authors: Patrick Lewis and co-authors.

Year: 2020.

Method: Retrieval-Augmented Generation combines a language model with retrieved information from an external knowledge source.

Application to Travel Pro: A similar architecture could retrieve verified destination descriptions, attraction details, travel guidelines, or seasonal information before generating a response.

Limitation: Retrieval quality depends on the relevance, completeness, and freshness of the underlying knowledge base.

4. Google Maps Platform Documentation

Organization: Google.

Type: Official technical documentation.

Method: Google Maps Platform provides mapping, place information, geocoding, and route-related capabilities through its APIs and services.

Application to Travel Pro: Google Maps integration helps users visualize geographical locations and explore routes or destinations.

Limitation: Available features depend on the APIs enabled, permissions, API keys, service availability, and applicable usage limits.

5. LangGraph Persistence Documentation

Organization: LangChain.

Type: Official technical documentation.

Method: LangGraph documents persistence mechanisms for maintaining short-term conversation state and longer-term application memory.

Application to Travel Pro: These concepts can support retaining travel preferences, continuing planning tasks, and resuming interrupted workflows if such mechanisms are implemented.

Limitation: Persistence requires appropriate configuration and storage. It is not automatically provided merely by displaying a conversation interface.

2.3 Literature Comparison Table

Author / Source	Year	Method	Application	Advantages	Limitations
Yao et al., ReAct	2023	Reasoning combined with actions	Tool-using AI agents	Supports iterative task execution	Requires reliable tools and validation
Schick et al., Toolformer	2023	Language-model tool use	External API integration	Connects language models with specialized tools	Tool failures and incorrect parameters remain possible
Lewis et al., RAG	2020	Retrieval-augmented generation	Knowledge-intensive responses	Can ground responses in retrieved information	Depends on retrieval and knowledge-base quality
Google Maps Platform	Official documentation	Mapping and geospatial APIs	Destination visualization and navigation	Provides geographical context	API access and usage constraints apply
LangGraph	Official documentation	State persistence and memory	Conversation continuity and agent workflows	Supports resumable workflows	Requires explicit implementation and configuration

2.4 Research Gap

The reviewed approaches address individual aspects of intelligent systems, such as reasoning, tool usage, knowledge retrieval, geographical services, and memory.

A travel planning application must coordinate these capabilities around a single user goal. For example, a traveler may need to identify a destination, select an appropriate travel month, create an itinerary, examine the location on a map, and consider weather conditions.

The proposed project brings destination discovery, trip planning, seasonal exploration, maps, and weather information together in one web application.

The research gap addressed by the project is the need for an integrated travel planning experience that combines these different information sources and planning activities in a user-friendly interface.

The extent to which the application autonomously decomposes tasks or coordinates multiple agents must be established from the actual implementation.

2.5 References Used in the Literature Survey

1. Yao, S., et al. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." International Conference on Learning Representations. https://arxiv.org/abs/2210.03629
2. Schick, T., et al. (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools." Advances in Neural Information Processing Systems 36. https://arxiv.org/abs/2302.04761
3. Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in Neural Information Processing Systems 33. https://arxiv.org/abs/2005.11401
4. Google. "Google Maps Platform Documentation." https://developers.google.com/maps/documentation
5. LangChain. "LangGraph Persistence." https://langchain-ai.github.io/langgraph/concepts/durable_execution/

CHAPTER 3: PROBLEM DEFINITION AND REQUIREMENTS

3.1 Existing System

Traditional travel planning involves using separate applications or websites for destination research, maps, weather information, seasonal recommendations, and itinerary preparation.

Users must often transfer information manually between these services. They may also need to compare multiple sources before deciding where to travel and how to organize their journey.

Some travel applications provide destination listings, while others focus on maps, weather, accommodation, or bookings. The planning experience can therefore become fragmented when these functions are not available together.

3.2 Limitations of the Existing System

The main limitations addressed by the proposed application are:

- Travel information may be distributed across multiple platforms.
- Seasonal destination discovery can require additional research.
- Users may need to switch between travel websites and mapping services.
- Weather conditions may not be considered during initial trip planning.
- Creating an itinerary manually can require significant effort.
- Destination discovery and itinerary preparation may be handled separately.
- Incomplete user preferences can make it difficult to prepare a suitable trip.

These points describe general limitations of fragmented travel-planning workflows, not a claim that every existing travel application has all these problems.

3.3 Proposed System

Travel Pro Agentic AI is a web-based travel discovery and planning application focused on destinations across India.

The application provides an Adventure Builder, dedicated sections for states and union territories, a Smart Months Calendar, Google Maps integration, weather information, and login and registration interfaces.

The system is designed to make it easier for users to discover destinations, explore seasonal travel opportunities, create trip plans, and consider geographical and weather-related information.

From an agentic AI perspective, the intended workflow can be represented as:

User Goal → Planning → Information Gathering → Tool Usage → Trip Preparation → Validation → Final Itinerary

This workflow should be described as an implemented agentic workflow only if the source code confirms that the corresponding steps are executed by the application.

3.4 Functional Requirements

The application should provide the following functional capabilities:

1. Display the Travel Pro home page.
2. Provide navigation to the Adventure Builder.

3. Allow users to explore Indian states.
4. Allow users to explore union territories.
5. Provide a Smart Months Calendar for seasonal discovery.
6. Integrate Google Maps for destination-related geographical information.
7. Display weather information through the configured weather service.
8. Provide login and registration interfaces.
9. Accept travel-planning inputs supported by the application.
10. Display generated trip plans or itineraries when requested.
11. Handle incomplete or invalid inputs appropriately.
12. Display an appropriate message when an external service is unavailable.

3.5 Non-Functional Requirements

Usability: The interface should be easy to navigate and understandable to travelers.

Performance: Pages and trip-planning operations should respond within a reasonable time under normal network conditions.

Reliability: The application should handle API failures and invalid inputs without crashing.

Security: Authentication credentials and API keys should be handled securely.

Maintainability: Components and integrations should be organized so that the application can be updated.

Compatibility: The deployed application should work on supported modern web browsers.

Scalability: The design should permit additional destinations, tools, and travel-planning features to be incorporated.

3.6 Hardware Requirements

Component	Minimum development requirement
Processor	Modern dual-core processor or better
RAM	4 GB minimum; 8 GB recommended
Storage	Sufficient space for source code and development tools

Internet	Required for deployment and external API integrations
Display	Standard laptop or desktop display

These are suggested development requirements, not measured production-server specifications.

3.7 Software Requirements

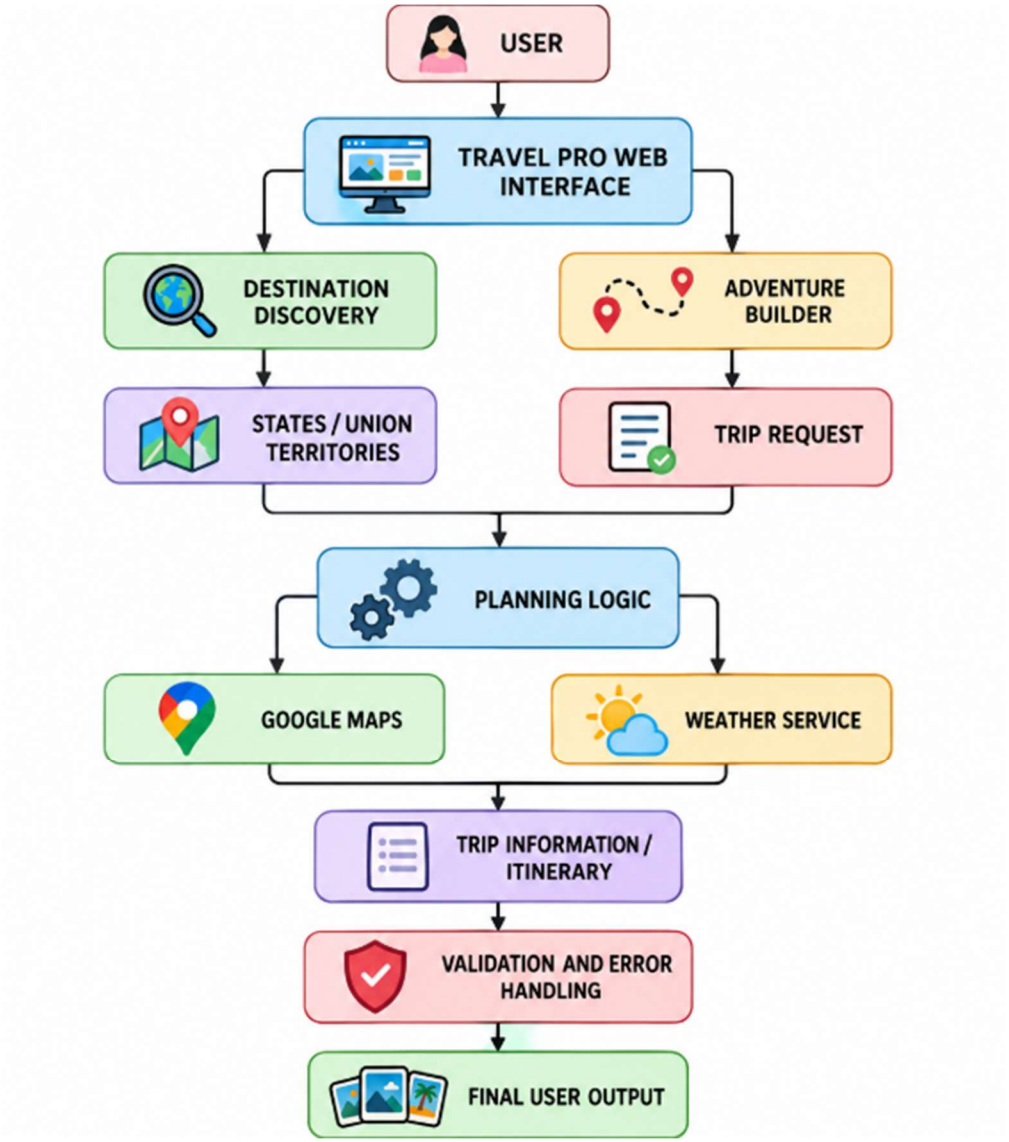
Software / Service	Purpose
Modern web browser	Access and test the application
Code editor	Develop and maintain source code
Git and GitHub, if used	Source-code version control
Vercel	Host the deployed frontend
Google Maps Platform	Map-related integration
Weather service/API	Weather information
LLM provider	Required if AI generation is implemented
Agent framework	Required if an external agent framework is used
Database	Required if accounts, preferences, or itineraries are persisted

Note: The exact programming language, frontend framework, LLM provider, agent framework, database, and weather provider must be filled in from the actual source code and project configuration.

CHAPTER 4: SYSTEM DESIGN

4.1 Overall Architecture

The proposed architecture describes how the user interface, travel-planning process, external services, and output validation can work together.



An LLM, persistent memory, RAG, and multiple specialized agents should be added to this diagram only when confirmed by the implementation.

4.2 Agent Architecture

An agentic version of the travel-planning workflow can be organized into the following logical responsibilities:

1. Planner / Supervisor: Interprets the user's travel goal and identifies the tasks required to prepare a plan.

2. Destination Researcher: Gathers relevant information about destinations and attractions.

3. Seasonal Analysis Component: Considers the requested month and available seasonal information.

4. Map and Weather Tool Component: Retrieves or displays geographical and weather information through the configured external services.

5. Itinerary Preparation Component: Organizes the gathered information into a travel plan.

6. Validation Component: Checks whether required inputs are available and whether the proposed output is consistent with the retrieved information.

These are proposed logical roles for explaining the design. They must not be presented as separate implemented AI agents unless the source code demonstrates their execution and communication.

4.3 Agent Roles and Responsibilities

Component	Responsibility	Expected output
User Interface	Collects user requests	Travel preferences
Planner	Determines required planning steps, if implemented	Task plan
Destination Component	Provides destination-related information	Destination details
Seasonal Component	Supports month-based discovery	Seasonal suggestions
Map Integration	Presents geographical information	Map or location view
Weather Integration	Retrieves weather information	Weather data

Itinerary Component	Organizes travel details	Proposed itinerary
Validation Component	Checks inputs and service responses	Validated or corrected output

4.4 Task Planning and Decomposition

A multi-step travel request can be divided into smaller tasks.

For example, a request to plan a four-day trip to Kerala during December could be decomposed into:

1. Identify the destination and travel duration.
2. Collect additional preferences, if required.
3. Identify suitable locations and attractions.
4. Consider seasonal information for December.
5. Retrieve relevant map and weather information.
6. Organize the proposed itinerary by day.
7. Validate the availability of required information.
8. Present the final trip plan.

The actual application should execute the applicable steps through its implemented logic. A static list of steps alone does not establish autonomous planning.

4.5 Tool Integration

Google Maps Integration

Google Maps is integrated into the application to support geographical exploration and destination-related navigation.

Its role may include displaying destination locations, maps, or route-related information, depending on the APIs and functionality configured in the project.

Official documentation: https://developers.google.com/maps/documentation

Weather Integration

The application provides live weather information to help travelers consider weather conditions when planning a journey.

The exact weather provider, forecast period, supported locations, and returned fields depend on the configured weather service.

Weather data should be described as current conditions or forecasts according to the actual API response, rather than as a guaranteed prediction.

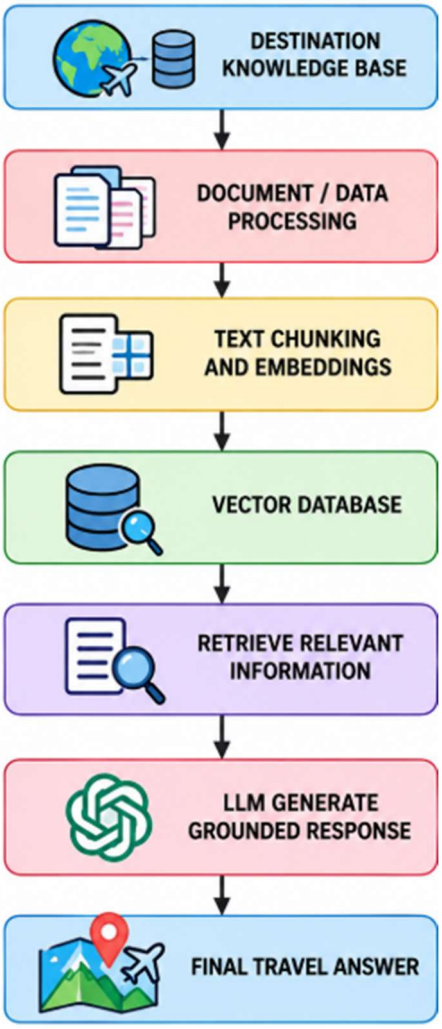
Additional Tools

The application may also use internal destination data, browser navigation, authentication services, a database, or an LLM API. These should be documented only when confirmed in the source code.

4.6 RAG Architecture

Retrieval-Augmented Generation combines retrieval from a knowledge source with language-model generation.

For a travel application, a potential RAG workflow is:



This architecture could retrieve destination guides, attraction descriptions, seasonal recommendations, or travel rules.

However, the existence of a RAG system, embedding model, or vector database in Travel Pro has not been confirmed. If these components are not implemented, this subsection should state that RAG is not currently implemented and discuss it as future work.

4.7 Memory Architecture

Memory in an agentic system can retain relevant context across planning steps or conversations.

For example, useful travel preferences could include:

- Preferred destinations.
- Travel duration.
- Budget range.
- Preferred travel activities.
- Number of travelers.
- Previously selected locations.

Short-term memory can preserve information during a planning session, while persistent memory can store preferences for later sessions.

The implementation must specify whether these values are retained only in the browser, in application state, or in a database. Login and registration pages alone do not prove that persistent travel-preference memory is available.

4.8 Agent Communication

When multiple agents are implemented, they must exchange information through a defined mechanism.

For example:

Planner → Destination Researcher → Seasonal Analysis → Map and Weather Component → Itinerary Component → Validator

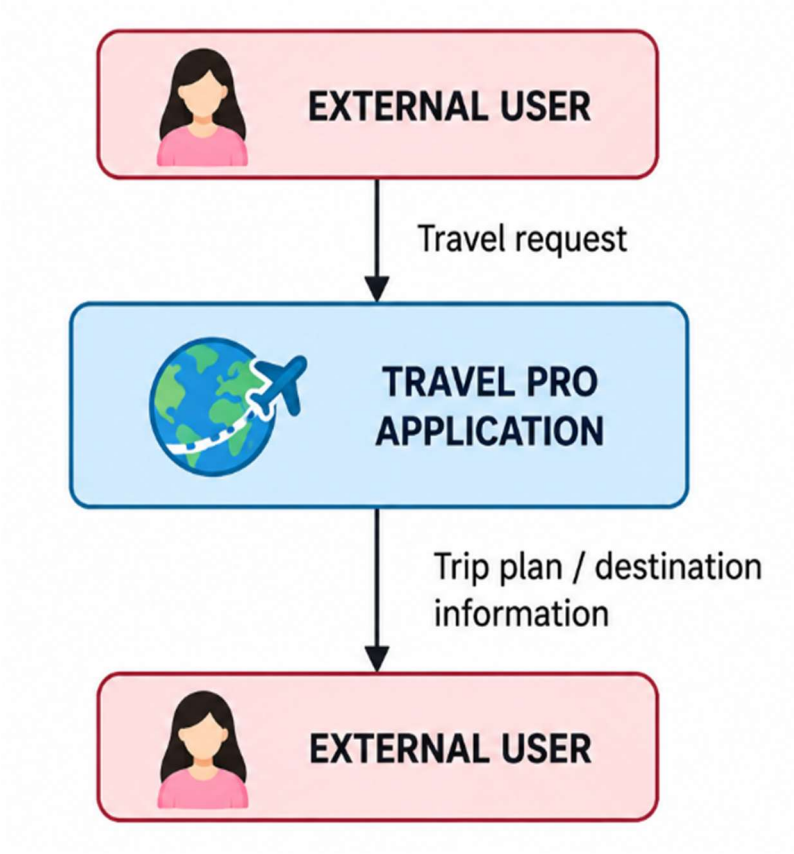
The planner could provide destination and duration requirements to the research component. The research component could return relevant locations, while the weather component could return weather information for the itinerary component to consider.

The validator could then check whether the required information was obtained before the result is presented.

This is a proposed communication design. If the existing application uses a single planning function rather than separate agents, the report should describe that implementation accurately instead.

4.9 Data Flow Diagram

Level 0: Context Diagram

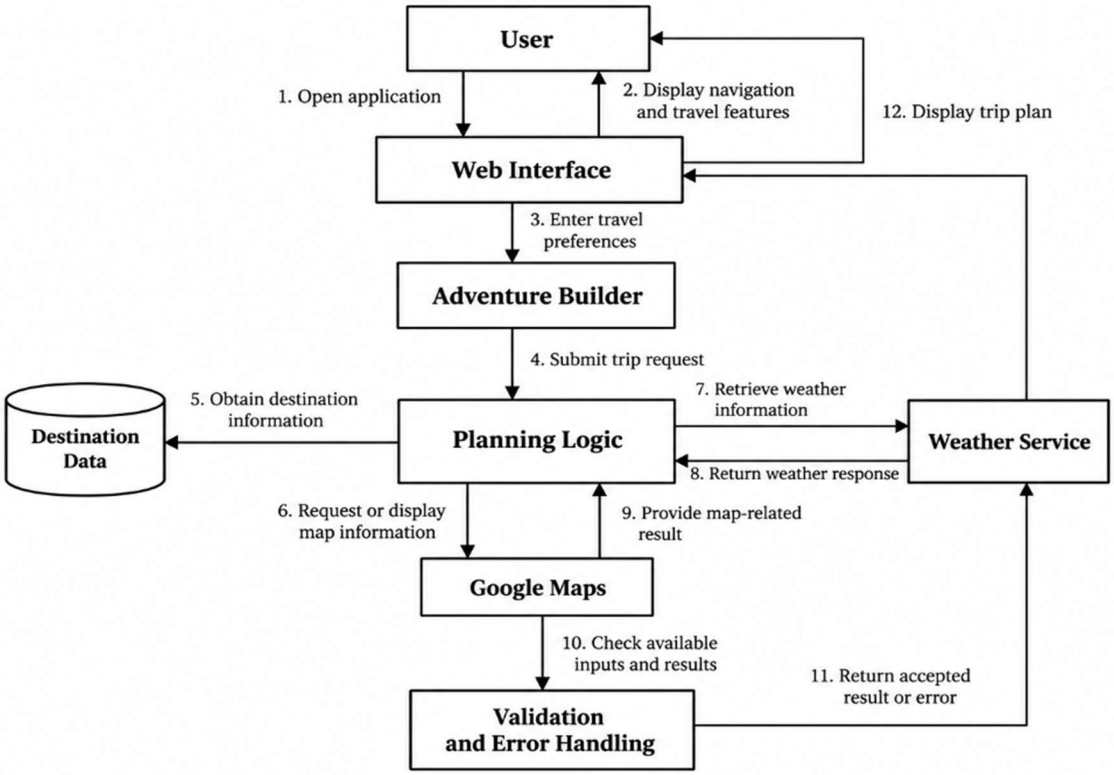


The application also communicates with external mapping and weather services where required.

Level 1: Detailed Data Flow

1. The user opens the application.
2. The user explores a destination or enters trip-planning details.
3. The application processes the request.
4. Destination information is retrieved or selected.
5. Map and weather services are used where applicable.
6. The planning logic organizes the available information.
7. The application handles errors or missing information.
8. The final result is displayed to the user.

4.10 Sequence Diagram



The actual sequence will vary according to the implementation and the way the map and weather services are called.

4.11 Use Case Diagram

Primary Actor: Traveler

Use Cases:

- Open the home page.
- Explore states.
- Explore union territories.
- Discover destinations by month.
- Create a trip plan.
- View geographical information.
- Check weather information.
- Register an account.
- Log in to an account.

A traveler interacts with the web application to select features and provide trip details. The application processes the request and displays the available information.

CHAPTER 5: IMPLEMENTATION

5.1 LLM Integration

A large language model can interpret natural-language travel requests, identify relevant preferences, and generate readable itineraries.

For the final report, the LLM provider, model identifier, API integration method, and configuration must be recorded from the actual project.

If the deployed application uses an LLM API, the report should explain how user inputs are passed to the model and how the returned response is displayed. If the project does not use an LLM, it should not claim LLM integration merely because it generates travel-related output.

5.2 Prompt Design

A travel-planning prompt can instruct an LLM to organize an itinerary according to user preferences.

An illustrative prompt is:

“You are a travel-planning assistant specializing in destinations across India. Use the user's destination, travel dates, duration, budget, and interests to prepare an organized itinerary. Consider available destination information, map results, and weather data when provided. Do not invent API results. If essential information is missing, request clarification. Clearly identify estimates and unavailable information.”

This is an example of prompt design, not a claim that the deployed application uses this exact prompt.

5.3 Agent Implementation

The implementation should explain the actual planning mechanism.

If separate agents are implemented, document their names, responsibilities, inputs, outputs, and execution order.

If the application uses a single AI planning component, document that component and explain how it invokes external tools.

A component should be classified as an AI agent only when its actual behavior supports the claim. Ordinary website pages, API calls, and navigation links are useful application components but do not independently establish an AI agent.

5.4 Planning Mechanism

The planning mechanism should process the user request and identify the information needed to create a suitable itinerary.

Depending on the implemented logic, it may use:

1. User destination and travel duration.
2. Month or travel dates.
3. Interests and preferences.
4. Destination information.
5. Map and weather information.
6. Itinerary organization.
7. Output validation.

The system should ask for missing essential inputs or use explicitly documented defaults rather than silently inventing values.

5.5 Tool Integration

Google Maps

Google Maps integration provides geographical context for travel locations. The user can use the available map interface to explore destinations or navigation information supported by the application.

Weather Service

The weather integration supplies weather information for supported locations. The application can use this information to help travelers consider weather when reviewing a trip.

Error Handling

External service integrations should account for:

- Missing or invalid API keys.
- Network connection failures.
- Invalid location parameters.

- Rate limits.
- Unavailable or incomplete responses.
- Unexpected API response formats.

The application should display an appropriate message and avoid presenting unavailable information as verified data.

5.6 RAG Implementation

If the source code contains a retrieval pipeline, document the knowledge sources, chunking strategy, embedding model, vector database, retrieval method, and how retrieved content is supplied to the LLM.

If no retrieval pipeline exists, record RAG as a current limitation and identify it as a possible enhancement.

5.7 Memory Implementation

The report should explain where conversation history or travel preferences are stored and how they are reused.

Possible mechanisms include browser state, session storage, local storage, or server-side database persistence. These mechanisms are not interchangeable, and the report must name only the one actually implemented.

If the application does not retain context between requests or sessions, that should be documented as a limitation.

5.8 Multi-Agent Communication

If multiple AI agents are used, describe the communication mechanism, such as shared state, function calls, structured messages, or an agent orchestration framework.

Include a sample input and output for each agent, if available.

If no separate agents communicate with one another, describe the current planning flow without claiming multi-agent execution.

5.9 Error Handling

The application should handle the following conditions:

- Empty trip-planning input.
- Invalid dates or travel duration.
- Missing destination information.

- Weather service timeout.
- Map service loading failure.
- Authentication errors.
- Network disconnection.
- Invalid or incomplete external API responses.

The expected behavior is to provide a clear message, allow correction where possible, and avoid returning misleading information.

5.10 User Interface

The deployed application contains the following main navigation features:

Home Page: The main entry point for the travel discovery experience.

Adventure Builder: The trip-planning interface, available at /create-trip.

States: The section for exploring Indian states, available at /states.

Union Territories: The section for exploring union territories, available at /union-territories.

Smart Months Calendar: The section for month-based seasonal discovery, available at /smart-calendar.

Login: The account login interface, available at /login.

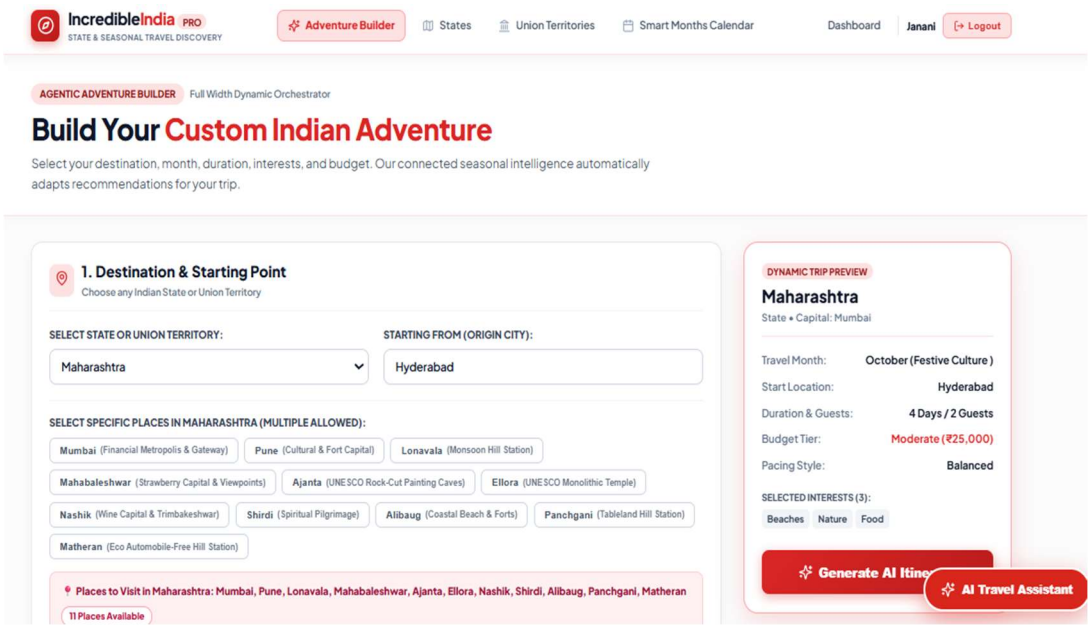
Register: The account registration interface, available at /register.

Google Maps: The integrated map experience for geographical exploration.

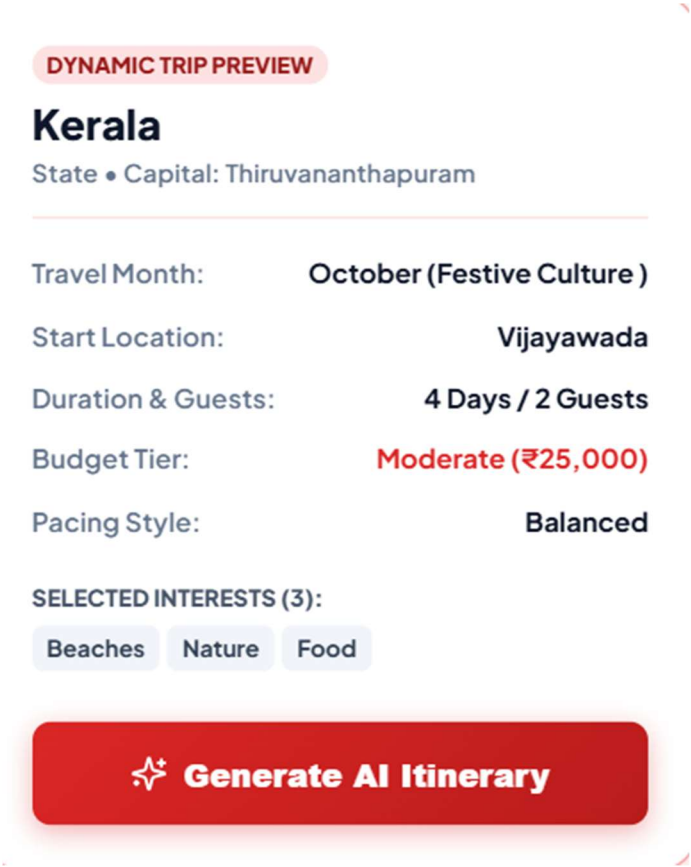
Weather Information: The weather feature that supports travel planning.

Deployment URL: https://travel-pro-agentic-ai-umber.vercel.app/

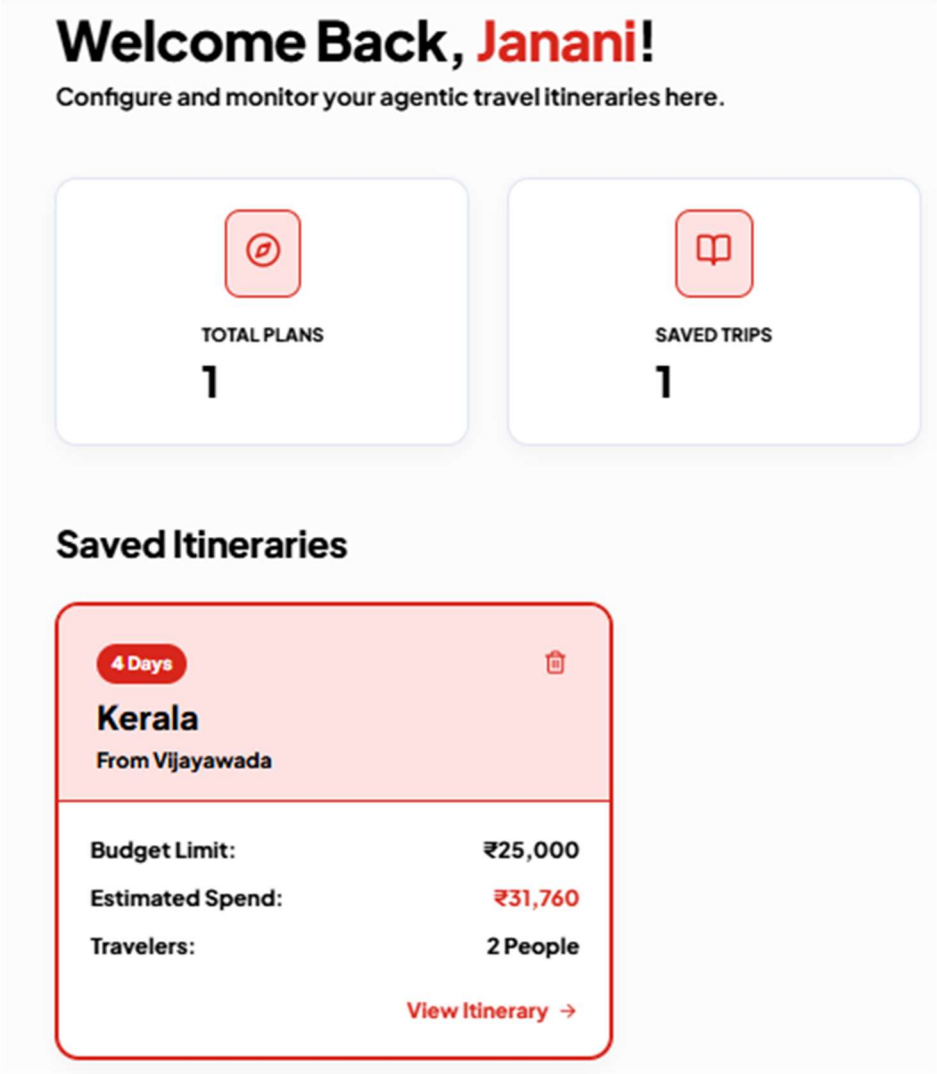
- Figure 5.1: Travel Pro home page.



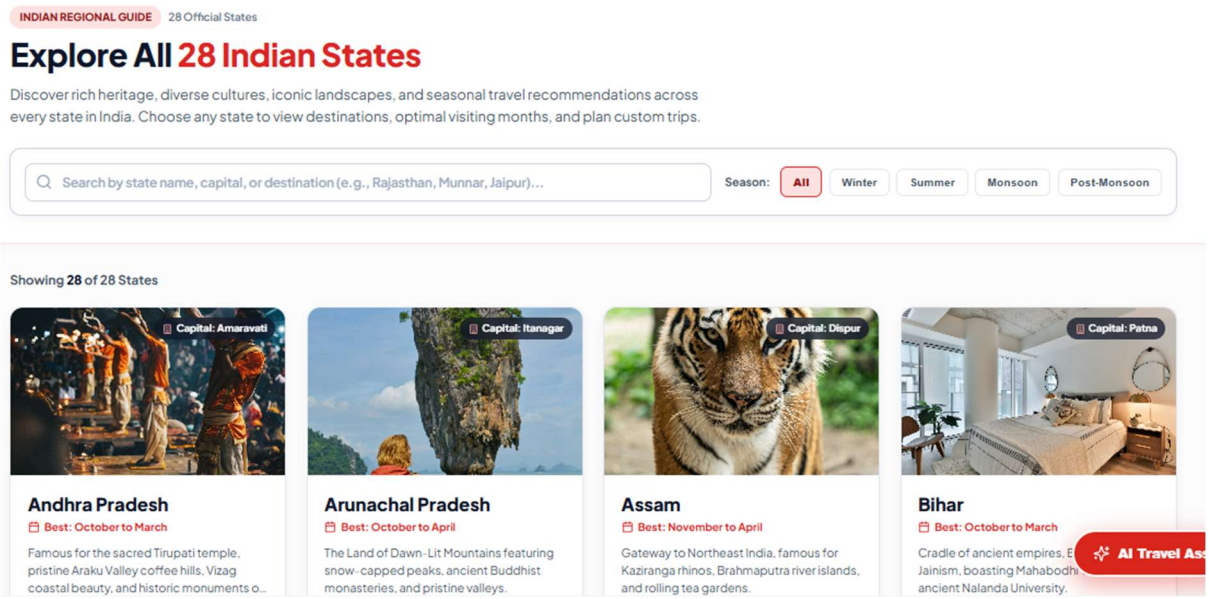
- Figure 5.2: Adventure Builder form.



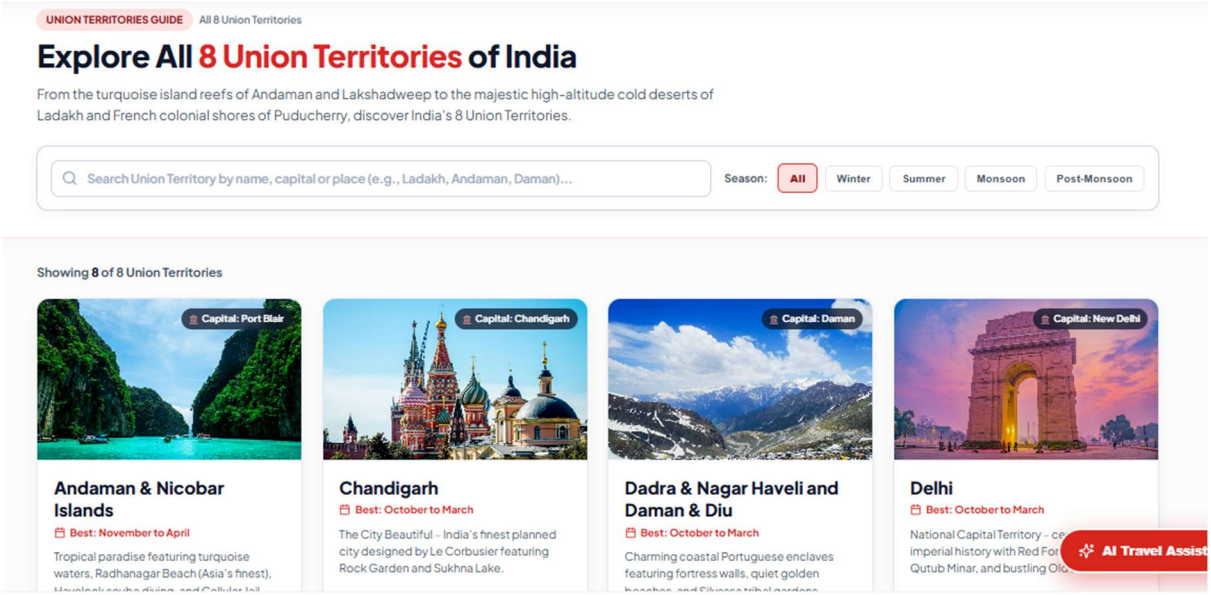
- Figure 5.3: Generated trip itinerary.



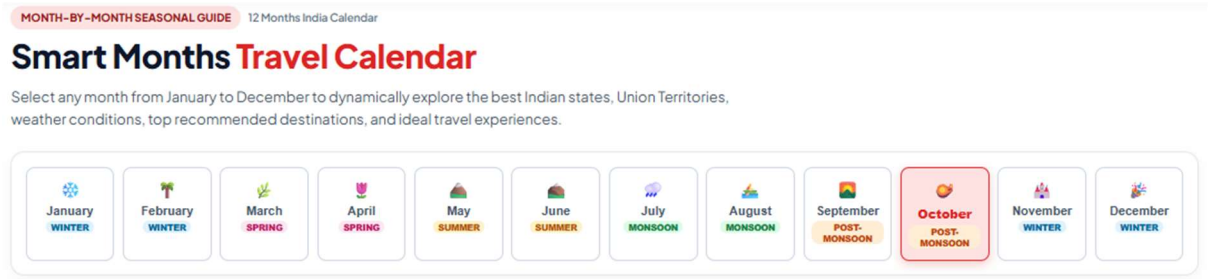
- Figure 5.4: States exploration page.



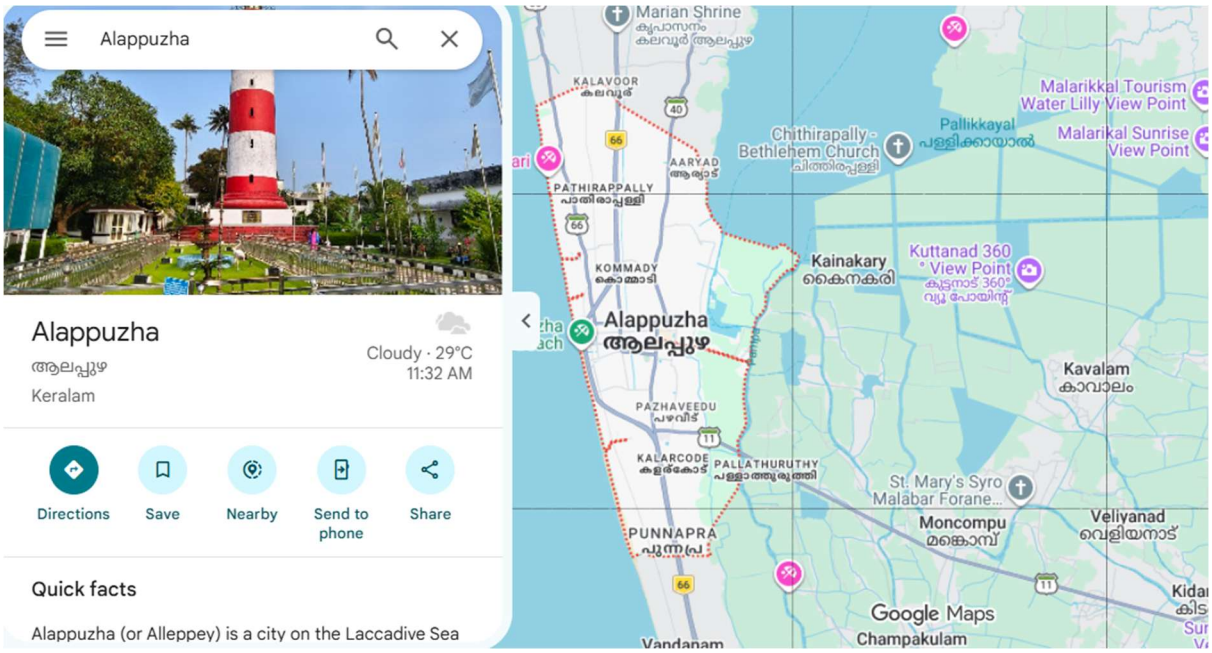
- Figure 5.5: Union Territories page.



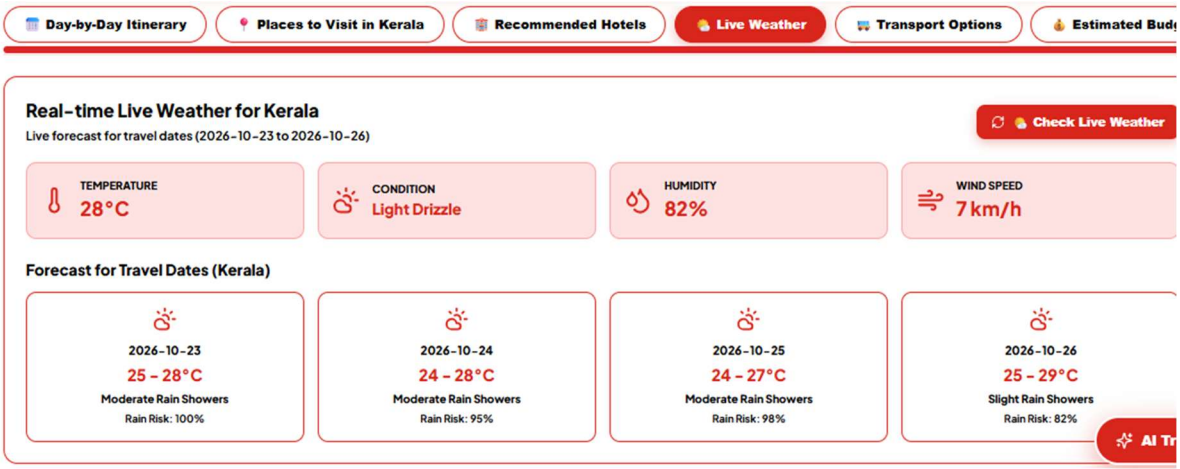
- Figure 5.6: Smart Months Calendar.



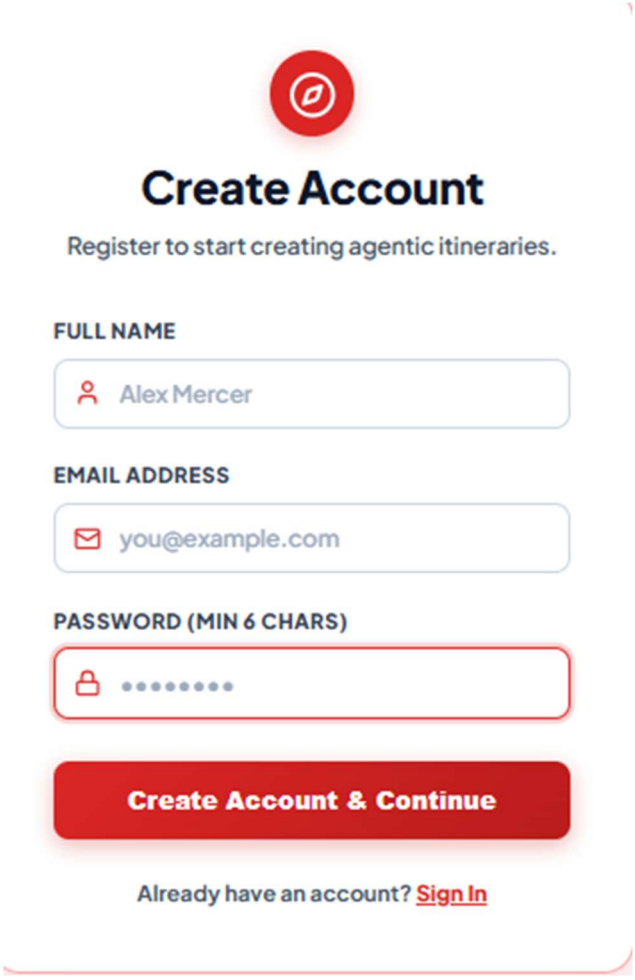
- Figure 5.7: Google Maps integration.



- Figure 5.8: Weather information displayed by the application.



- Figure 5.9: Login or registration interface.



CHAPTER 6: TESTING AND RESULTS

6.1 Introduction

Testing is required to verify that the application's navigation, trip-planning features, external integrations, and error handling work as intended.

Tests should cover normal inputs, multiple-step trip requests, map and weather integration, invalid inputs, and unavailable services.

The actual output and test status must be recorded after executing the corresponding tests.

6.2 Test Cases

Test Case	Input / Action	Expected Output	Actual Output	Status
TC01	Open the home page	Home page loads	Record after testing	Pending
TC02	Open Adventure Builder	Trip-planning interface appears	Record after testing	Pending
TC03	Submit a valid trip request	Trip plan is displayed	Record after testing	Pending
TC04	Explore the States page	State information is displayed	Record after testing	Pending
TC05	Open Union Territories	Union Territories section loads	Record after testing	Pending
TC06	Select a month in Smart Months Calendar	Relevant seasonal information appears	Record after testing	Pending
TC07	Open map information for a destination	Map or location information is displayed	Record after testing	Pending
TC08	Request weather information	Weather data is displayed if available	Record after testing	Pending
TC09	Submit an empty trip request	Validation or a suitable prompt is displayed	Record after testing	Pending

TC10	Simulate an unavailable weather service	A suitable error or fallback is displayed	Record after testing	Pending
TC11	Open the login page	Login interface loads	Record after testing	Pending
TC12	Open the registration page	Registration interface loads	Record after testing	Pending

The status should be changed to Pass only after the expected behavior has been observed. Failed tests should be documented with their actual results and corrective action.

6.3 Results and Discussion

The application brings together destination discovery, trip planning, seasonal exploration, mapping, and weather information in a common interface.

Its principal user-facing features can be evaluated by observing whether users can navigate between sections, create trip plans, explore destinations, and access external information successfully.

A complete results section should include evidence from the deployed application and report any failures or limitations found during testing.

6.4 Performance Evaluation

The following metrics are suitable for evaluating the application.

Task Success Rate

Task success rate measures the proportion of test tasks completed successfully.

Task Success Rate (%) = (Successfully completed tasks / Total tested tasks) × 100

Response Time

Response time measures the time between submitting a request and receiving the expected result.

Record multiple observations for trip generation, map loading, and weather retrieval where applicable.

Tool Success Rate

Tool success rate measures how many external tool calls complete successfully.

Tool Success Rate (%) = (Successful tool calls / Total tool calls) × 100

Response Accuracy

Response accuracy measures whether the generated itinerary and associated information satisfy the specified requirements and agree with available evidence.

Error Handling Rate

Error handling rate measures the proportion of tested failure conditions that are handled appropriately.

Evaluation Table

Metric	Measurement Method	Result
Task Success Rate	Successful tasks divided by tested tasks	To be measured
Response Time	Timed test runs	To be measured
Tool Success Rate	Successful API calls divided by total calls	To be measured
Response Accuracy	Manual review against requirements and available evidence	To be measured
Error Handling Rate	Correctly handled failure cases divided by tested failures	To be measured
User Satisfaction	Feedback from test users	To be measured

No numerical performance claims should be included until measurements have been collected.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

Travel Pro Agentic AI is a web-based application developed to support travel discovery and trip planning across India.

The application provides an Adventure Builder, destination exploration through States and Union Territories sections, a Smart Months Calendar, Google Maps integration, weather information, and login and registration interfaces.

By bringing these capabilities together, the application aims to simplify the process of exploring travel destinations and preparing trip plans. Its integration of geographical and weather information gives users additional context when evaluating potential journeys.

The project provides a foundation for incorporating more advanced agentic AI capabilities, including structured task planning, tool orchestration, persistent memory, retrieval-augmented generation, and multi-agent collaboration.

The final assessment of its agentic AI functionality depends on the planning, tool-calling, and validation mechanisms confirmed in the implementation.

7.2 Limitations

Potential limitations to document after examining and testing the application include:

- Dependence on internet connectivity for external services.
- Dependence on third-party map and weather service availability.
- Weather information may be unavailable for certain locations or time periods.
- Generated itineraries may require manual review.
- Destination information may need periodic updates.
- Persistent travel-preference memory may be limited or unavailable.
- RAG and multi-agent orchestration may not be implemented.
- External API usage may be subject to rate limits and costs.
- The accuracy of recommendations depends on the information available to the application.

Only limitations that apply to the actual implementation should be retained in the final report.

7.3 Future Scope

The following improvements could extend the project:

1. Implement a dedicated AI planner that decomposes complex travel requests into subtasks.
2. Add specialized destination research, seasonal analysis, and itinerary validation agents.
3. Introduce RAG using verified destination guides and tourism documents.
4. Add persistent memory for user preferences and previously created itineraries.
5. Integrate additional APIs for transport, accommodation, and tourism information.
6. Provide budget estimation and expense tracking.
7. Add route optimization for multiple destinations.
8. Introduce weather-aware itinerary adjustments.
9. Provide downloadable PDF itineraries.
10. Add voice-based trip planning.
11. Introduce multilingual travel assistance.
12. Add human approval before executing important external actions.
13. Improve monitoring, logging, testing, and API-failure recovery.
14. Expand the destination knowledge base with verified and regularly updated information.

CHAPTER 8: REFERENCES

1. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. "ReAct: Synergizing Reasoning and Acting in Language Models." International Conference on Learning Representations, 2023. https://arxiv.org/abs/2210.03629
2. Schick, T., et al. "Toolformer: Language Models Can Teach Themselves to Use Tools." Advances in Neural Information Processing Systems 36, 2023. https://arxiv.org/abs/2302.04761
3. Lewis, P., et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in Neural Information Processing Systems 33, 2020. https://arxiv.org/abs/2005.11401
4. Google. "Google Maps Platform Documentation." https://developers.google.com/maps/documentation
5. LangChain. "LangGraph Persistence." https://langchain-ai.github.io/langgraph/concepts/durable_execution/
6. Google. "Google Maps Platform, Maps JavaScript API." https://developers.google.com/maps/documentation/javascript
7. [Weather API Provider]. "[Exact API documentation title]." [Insert the official documentation URL for the weather service actually used.]
8. [LLM Provider]. "[Exact model or API documentation title]." [Insert the official documentation URL for the LLM actually used, if applicable.]
9. [Agent Framework]. "[Exact framework documentation title]." [Insert the official documentation URL for the framework actually used, if applicable.]
10. [Frontend Framework / Hosting Platform]. "[Relevant official documentation title]." [Insert the documentation URLs for the actual framework and hosting platform.]

The final reference list should contain only the sources used in the report. Replace the bracketed entries with the exact services and versions found in the project's source code.